

COGNIVIEW

Real-Time AI-Powered Surveillance System

Product Requirements Document (PRD)

Document Version: v1.0

Classification: Student Prototype — Internal Use

Author: Project Team

Status: Draft

Date: 2025

Tech Stack: Python · OpenCV · YOLOv8

CONFIDENTIAL — PROTOTYPE LEVEL DOCUMENTATION

1. Problem Statement

1.1 The Challenge of Manual Video Surveillance

Modern surveillance environments generate an overwhelming volume of video data across multiple camera feeds simultaneously. Human operators tasked with monitoring these feeds face serious cognitive and operational limitations.

Key Challenges

- **Operator Fatigue**
 - Sustained attention is physiologically unsustainable over long monitoring shifts.
 - Studies indicate focus and reaction accuracy degrade sharply after 20–30 minutes of continuous screen monitoring.
- **Alert Fatigue**
 - Legacy motion-detection systems generate a high ratio of false positives triggered by environmental factors such as lighting changes, moving shadows, and animals.
 - Operators eventually become desensitised to alarms, causing genuine threats to be ignored.
- **Delayed Response**
 - Manual identification of intrusion events is inherently reactive rather than preventive.
 - Human review of recorded footage after an incident provides no real-time intervention capability.
- **Scalability Bottleneck**
 - Adding camera feeds increases the monitoring burden linearly, making large-scale manual surveillance impractical.

1.2 The Need for Automated Real-Time Monitoring

Automated systems can process video feeds continuously, without fatigue, and flag only meaningful events for human review. By integrating AI-based object detection with rule-based intrusion logic, it is possible to build a system that:

- Operates 24/7 without degradation in attention
- Applies consistent detection thresholds, eliminating subjective human variance
- Generates timestamped, image-backed alert records for audit and review
- Allows operators to focus exclusively on verified alerts rather than raw footage

1.3 Risks of Current Approaches

Missed threats and alert fatigue represent the two ends of the surveillance failure spectrum. COGNIVIEW is designed to address both.

- **Missed Threats:** Undetected intrusions into restricted zones lead to security breaches, property damage, or safety incidents.

- Alert Overload: Systems with poor filtering create noise that erodes operator trust and responsiveness.
- Lack of Evidence Trail: Without automated image capture and event logging, post-incident investigation becomes difficult.

2. Goals & Objectives

COGNIVIEW is designed as a realistic, prototype-level AI surveillance system. The following goals define what the project aims to achieve at this stage.

2.1 Real-Time Human Detection

- Detect human figures in live webcam feeds or pre-recorded video files with minimal latency.
- Draw bounding boxes around detected persons with confidence scores overlaid on the video frame.
- Achieve real-time or near-real-time processing on standard consumer hardware (CPU-based).

2.2 Intrusion Detection in Restricted Zones

- Allow users to define one or more Regions of Interest (ROIs) on the video frame.
- Trigger intrusion events whenever a detected person's bounding box overlaps with a defined ROI.
- Support flexible rectangular ROI definitions via configuration or user input.

2.3 Reduction of False Alerts

- Apply confidence score thresholds to discard low-certainty detections.
- Use temporal filtering (frame-window validation) to confirm that a detection persists across multiple consecutive frames before triggering an alert.
- Prevent duplicate or redundant alerts for the same sustained intrusion event.

2.4 Lightweight and Deployable Prototype

- Build entirely in Python using open-source libraries (OpenCV, Ultralytics YOLOv8).
- Require no custom model training — leverage the pretrained COCO-based YOLOv8 model.
- Produce a GitHub-hosted prototype that peers, mentors, and evaluators can clone and run locally.
- Generate deliverables including: alert images, timestamped event logs, and test video results.

3. Scope Definition

Defining scope clearly is critical to maintaining project focus and avoiding scope creep in a student-built prototype.

3.1 In Scope

Feature	Description
YOLOv8 Human Detection	Detect persons in video using pretrained YOLOv8n/s model
ROI-Based Intrusion Logic	Geometric check for bounding box overlap within defined region
Confidence Filtering	Discard detections below configurable threshold (e.g. 0.5)
Temporal Filtering	Require detection to persist for N consecutive frames before alert
Alert Image Capture	Save annotated frame as PNG on intrusion confirmation
Event Logging	Write timestamped intrusion events to a text/CSV log file
OpenCV Visualisation	Live display window with bounding boxes, ROI overlay, and status
Webcam + Video File Support	Accept both live webcam feed and pre-recorded .mp4/.avi files

3.2 Out of Scope

The following capabilities are intentionally excluded from this prototype. They are noted as future enhancement opportunities where relevant.

Excluded Feature	Rationale
Facial Recognition	Raises significant privacy concerns; well beyond prototype scope
Person Re-Identification	Requires specialised multi-camera tracking models (e.g. Re-ID networks)
Multi-Camera Support	Not required at prototype level; identified as future enhancement
Drone / PTZ Camera Control	Hardware integration beyond software project boundaries
Cloud / Remote Deployment	Network infrastructure and security requirements not in scope
Custom Model Training	Inference-only approach; training requires large labelled datasets
Web Dashboard / Mobile App	UI development scope deferred to future iterations
Audio-Based Detection	Outside the video-only detection scope of this system

4. System Overview & Architecture

COGNIVIEW follows a modular pipeline architecture where each stage performs a single, well-defined function. This design promotes testability, maintainability, and easy debugging.

4.1 High-Level Pipeline

Video Input → Preprocessing → Detection Engine → Intrusion Logic → Filtering → Alert System → Output Visualisation

4.2 Module Descriptions

Module 1 — Video Input Layer

- Accepts input from a USB/integrated webcam via OpenCV's VideoCapture(0).
- Alternatively reads a pre-recorded video file (e.g. .mp4, .avi) via VideoCapture(filepath).
- Outputs a continuous stream of raw BGR frames to the preprocessing module.

Module 2 — Preprocessing Layer

- Resizes frames to a consistent resolution (e.g. 640x480 or 1280x720) for inference.
- Converts BGR (OpenCV default) to RGB format as required by YOLOv8.
- Optionally applies histogram equalisation or brightness normalisation for low-light conditions.

Module 3 — Detection Engine (YOLOv8)

- Loads the pretrained YOLOv8n (nano) or YOLOv8s (small) model via Ultralytics API.
- Runs per-frame inference, returning bounding boxes, class labels, and confidence scores.
- Filters results to retain only detections of class ID 0 (person) from the COCO label set.

Module 4 — Intrusion Logic (ROI Engine)

- Reads one or more ROI definitions as pixel-coordinate rectangles (x1, y1, x2, y2).
- For each detected person, checks if the centre point or bounding box intersects the ROI.
- Flags the frame as a candidate intrusion event if the overlap condition is satisfied.

Module 5 — Filtering Layer

- Confidence Filter: Drops any detection with a score below the configured threshold.
- Temporal Filter: Maintains a per-ROI frame counter. An alert is triggered only after a person has been detected inside the ROI for a minimum of N consecutive frames (e.g. N=3).
- Cooldown Filter: Enforces a minimum time gap between successive alerts to prevent alert storms.

Module 6 — Alert System

- On confirmed intrusion: saves an annotated snapshot (bounding box + ROI overlay) as a PNG file with a timestamp in the filename.
- Appends an entry to an event log file including: timestamp, frame number, ROI name, confidence score.
- Optionally prints a console alert for real-time operator awareness.

Module 7 — Output Visualisation

- Uses OpenCV's imshow to render a live annotated video window.
- Overlays: bounding boxes (green/red), confidence labels, ROI boundary rectangle, and intrusion status text.
- Colour coding: Green = person detected outside ROI; Red = intrusion confirmed.

5. Functional Requirements

5.1 Video Input

Req ID	Requirement	Priority
FR-V01	System shall accept live webcam input via OpenCV VideoCapture(0).	Must Have
FR-V02	System shall accept pre-recorded video files (.mp4, .avi).	Must Have
FR-V03	System shall display the current frame rate in the visualisation window.	Should Have
FR-V04	System shall gracefully handle end-of-file for video inputs.	Must Have

5.2 Human Detection

Req ID	Requirement	Priority
FR-D01	System shall detect persons using a pretrained YOLOv8 model.	Must Have
FR-D02	System shall draw labelled bounding boxes around all detected persons.	Must Have
FR-D03	System shall display confidence scores on bounding box labels.	Must Have
FR-D04	System shall ignore all non-person COCO class detections.	Must Have
FR-D05	Detection confidence threshold shall be configurable (default: 0.5).	Must Have

5.3 Intrusion Detection

Req ID	Requirement	Priority
FR-I01	System shall support at least one user-defined rectangular ROI.	Must Have
FR-I02	ROI coordinates shall be configurable without code modification.	Should Have
FR-I03	System shall flag an intrusion when a person bounding box overlaps ROI.	Must Have
FR-I04	ROI boundary shall be rendered on the output video frame at all times.	Must Have

5.4 Filtering

Req ID	Requirement	Priority
FR-F01	Temporal filter shall require N consecutive frames before alert (default N=3).	Must Have
FR-F02	Alert cooldown shall prevent duplicate alerts within a configurable window.	Must Have
FR-F03	Confidence threshold and frame count shall be tunable via config file.	Should Have

5.5 Alert System

Req ID	Requirement	Priority
FR-A01	System shall save an annotated PNG image on each confirmed intrusion.	Must Have
FR-A02	Alert image filename shall include a UTC timestamp.	Must Have
FR-A03	System shall append an event record to a persistent log file (CSV/TXT).	Must Have
FR-A04	Log record shall include: timestamp, ROI name, confidence, frame number.	Must Have

6. Non-Functional Requirements

Category	Requirement	Target
Performance	Frame processing latency	< 100 ms per frame on CPU (≥10 FPS)
Accuracy	Person detection recall	Qualitatively acceptable on test videos

Category	Requirement	Target
Reliability	System uptime during test session	No crashes over a 10-minute continuous run
Usability	Setup and run time	Operable by a peer with Python knowledge in < 5 min
Portability	Cross-platform support	Windows and Linux (Python 3.9+)
Maintainability	Code modularity	Each pipeline stage in a separate function/class
Scalability	Future multi-camera support	Architecture must not preclude later extension
Lightweight	Model size	YOLOv8n (<10 MB) preferred for fast inference

7. Data Sourcing Strategy

This is an inference-first system. No custom model training is required. The data strategy focuses entirely on sourcing suitable test material to validate detection accuracy and filtering effectiveness.

7.1 Pretrained Model — COCO Dataset

YOLOv8 is pretrained on the COCO (Common Objects in Context) dataset, which provides robust out-of-the-box person detection capability.

- COCO contains over 330,000 images with 80 labelled object classes, including the 'person' class.
- The pretrained weights generalise well to indoor/outdoor surveillance scenarios without fine-tuning.
- No GPU or training infrastructure is required — weights are downloaded automatically by the Ultralytics library.

7.2 Public Datasets for Testing

VIRAT Video Dataset

- Source: viratdata.org
- Contains outdoor surveillance footage of people in parking lots, pathways, and open spaces.
- Ideal for testing multi-person detection, ROI entry/exit events, and long-duration monitoring.

UCF Crime Dataset

- Source: University of Central Florida
- Contains surveillance footage of anomalous scenarios including trespassing and loitering.
- Useful for testing edge cases such as fast movement, occlusion, and multiple simultaneous actors.

Open Images Dataset (Optional)

- Source: Google Open Images V7
- Can be filtered for person-class images to qualitatively validate bounding box accuracy.
- Useful for static frame validation if video datasets are unavailable.

7.3 Custom Data Collection

Beyond public datasets, the team should collect scenario-specific test videos using a standard webcam:

Recommended Scenarios

Scenario	Purpose
Single person entering ROI in normal light	Baseline intrusion detection validation
Multiple persons simultaneously in and around ROI	Test overlapping bounding box handling
Person moving quickly through ROI	Validate temporal filter is not too slow
Low-light environment (evening / dimmed room)	Test detection degradation under poor lighting
Motion blur from fast movement or camera shake	Evaluate robustness to image quality issues
Occluded person (partially behind furniture/door)	Test partial detection scenarios
Empty room with background motion (fan, curtains)	Validate false positive suppression

7.4 Data Usage Strategy

- Each public dataset or custom video is run through the full COGNIVIEW pipeline.
- False positives (non-person objects triggering alerts) are identified and recorded.
- Confidence thresholds and temporal frame counts are iteratively adjusted based on observed false-positive rates.
- At least two scenarios per lighting condition (normal, low-light) should be tested.
- Results are documented qualitatively in the evaluation section of the project report.

No annotation, labelling, or training pipeline is needed. All data is used purely for inference testing and system parameter tuning.

8. Model & Technical Approach

8.1 Detection Model — YOLOv8

Model Family	YOLOv8 by Ultralytics
Variant	YOLOv8n (nano) for CPU; YOLOv8s (small) optional for better accuracy
Trained On	COCO dataset (80 classes; class 0 = person)
Inference Mode	Per-frame real-time inference — no batch processing required
Customisation	None required at prototype level; fine-tuning is a future scope item
Integration	Loaded via Ultralytics Python API: model = YOLO('yolov8n.pt')

8.2 Video Processing — OpenCV

- Frame capture via cv2.VideoCapture — supports webcam index and file paths.
- Frame resizing via cv2.resize for consistent model input dimensions.
- Bounding box and ROI rendering via cv2.rectangle and cv2.putText.
- Alert image saving via cv2.imwrite with timestamped filenames.

8.3 ROI Intrusion Logic

ROI intersection is computed geometrically. For a detected person bounding box (px1, py1, px2, py2) and a defined ROI (rx1, ry1, rx2, ry2):

$$Intrusion = (px1 < rx2) \text{ AND } (px2 > rx1) \text{ AND } (py1 < ry2) \text{ AND } (py2 > ry1)$$

This axis-aligned bounding box overlap check is computationally trivial and requires no additional libraries.

8.4 Temporal Filtering

- A frame counter (intrusion_counter) is maintained per ROI.
- Counter increments on each frame where an intrusion is detected; resets to 0 when no intrusion is present.
- An alert is triggered only when intrusion_counter reaches the configured threshold (default: 3).
- A separate cooldown_counter prevents re-alerting for the same sustained intrusion event.

8.5 No Custom Training Required

The system relies entirely on YOLOv8's pretrained COCO weights. This is the correct approach for a prototype: the model already generalises well to person detection across a wide range of environments. Custom fine-tuning is documented as an optional future scope item.

9. Evaluation Metrics

Given the prototype scope, evaluation is primarily qualitative and scenario-based rather than large-scale statistical benchmarking.

Metric	Definition	Measurement Method	Target
Detection Recall	Fraction of visible persons correctly detected	Manual review of test videos	Qualitatively high (>80% of clear frames)
False Positive Rate	Alerts triggered without a real person in ROI	Count FP alerts across test scenarios	< 2 false alerts per 10-minute session
Alert Precision	Fraction of alerts that correspond to real intrusions	Manual audit of saved alert images	≥ 90% of saved images show real intrusion
System Latency	End-to-end frame processing time	cv2.getTickCount() timing	< 100 ms per frame (≥ 10 FPS)
Temporal Filter Efficacy	Reduction in spurious alerts vs. unfiltered baseline	Compare alert counts with/without filter	≥ 50% reduction in false alerts

Metrics are intentionally kept simple and observable. The goal is to demonstrate that filtering meaningfully improves system precision — not to report publication-grade benchmark numbers.

10. Risks & Mitigation

Risk	Impact	Likelihood	Mitigation Strategy
High false positive rate from background motion	Alert noise erodes trust	High	Tune confidence threshold; apply temporal filter; test on diverse scenes
Detection failure in low-light conditions	Missed intrusions	Medium	Pre-process with histogram equalisation; document as known limitation

Risk	Impact	Likelihood	Mitigation Strategy
Insufficient CPU performance for real-time	System unusable	Medium	Use YOLOv8n; reduce input resolution; test on target hardware early
Over-claiming system capabilities	Reputational / academic risk	Low	Clearly label as prototype; avoid defence/enterprise language in documentation
Dependency version conflicts	Environment setup failures	Medium	Pin all versions in requirements.txt; test on clean virtual environment
ROI coordinates not matching actual scene	Intrusion logic errors	Low	Provide a calibration utility or interactive ROI selection tool

11. Future Enhancements

The following enhancements are out of scope for the current prototype but represent a logical and achievable development roadmap for future iterations.

Enhancement	Description	Complexity
Object Tracking (DeepSORT)	Assign persistent IDs to tracked persons across frames; enable trajectory analysis	Medium
Multi-Camera Support	Process multiple camera feeds in parallel using threading or multiprocessing	Medium
Web Dashboard	Flask/FastAPI interface to display live feeds, alerts, and logs in a browser	Medium
Behaviour Anomaly Detection	Detect loitering, running, or unusual motion patterns using pose estimation	High
Edge Deployment (Raspberry Pi)	Optimise model for ARM hardware using ONNX or TensorRT export	High
Alert Notification System	Send email or Telegram notifications on confirmed intrusion events	Low
Custom Fine-Tuning	Fine-tune YOLOv8 on domain-specific datasets for improved accuracy in target scenes	High
Facial Recognition (Optional)	Integrate face identification for known-person whitelisting — requires careful privacy planning	Very High

12. Tech Stack

Component	Technology	Version	Role
Language	Python	3.9+	Primary development language for all modules
Video Processing	OpenCV (cv2)	4.8+	Frame capture, resize, drawing, and image saving
Object Detection	Ultralytics YOLOv8	8.x	Pretrained COCO model for person detection
Numerical Ops	NumPy	1.24+	Array operations, coordinate calculations
Logging	Python csv / logging	stdlib	Event log file writing
Config Management	Python json / yaml	stdlib	Externalise threshold and ROI configuration
Version Control	Git + GitHub	—	Source code management and project delivery

All dependencies are open-source and freely available via pip install. No paid APIs, cloud services, or proprietary SDKs are required.

13. Deliverables

Deliverable	Description	Format
Working Python Prototype	Runnable COGNIVIEW pipeline covering all modules in scope	Python scripts / package
GitHub Repository	Full source code with README, requirements.txt, and usage instructions	Public GitHub repo
Configuration File	Externalised config for ROI coordinates, thresholds, and model settings	config.yaml or config.json
Sample Test Videos	At least 3 test video clips demonstrating: normal detection, intrusion, and FP cases	.mp4 files
Alert Image Samples	Sample saved alert PNG images from confirmed intrusion events	PNG files in /alerts/
Event Log Sample	Sample CSV/TXT log file from a test run showing timestamped intrusion records	CSV or TXT file
Evaluation Report	Brief document summarising test scenarios, metrics observed, and known limitations	PDF or Markdown
Demo Video (Optional)	Screen-recorded walkthrough of the system running in real time	.mp4 screen capture